How to Use ADC in LPC1768? LPC1768 ADC Tutorial

March 28, 2024 By Ravi Teja(https://www.electronicshub.org/author/raviteja/)

In this tutorial, I will show you how to use the ADC in LPC1768, an Arm Cortex-M3 MCU. The ADC Peripheral is an important part of modern Microcontrollers as it helps us in interfacing the Analog Devices like Sensors to the MCU. This LPC1768 ADC Tutorial will be very useful if you are interested in interfacing analog sensors with the MCU.

IMPORTANT NOTE: I have mentioned this already in the previous tutorials but I will repeat it in every LPC1768 Tutorial. Download both the Datasheet and User Manual for LPC1768 MCU from the official NXP website. I cannot explain / discuss each and every topic in detail. You have to look up for the topic of discussion in those documents and gather additional information.

Outline

Introduction

ADC is short for Analog to Digital Converter (https://www.electronicshub.org/analog-circuits-and-digital-circuits/). As the name suggests, an ADC converts an analog signal to a digital signal. Why do we need such a conversion? To understand this, we have to first know what sort of signal are being converted.

ElectronicsHub
(https://www.electronicshub.org)

Take an example of an audio recording system. A Microphone is used to capture the sound. The sound is a continuous analog signal and if we want to store, edit or transfer it through digital systems like a Microprocessor or a Microcontroller, then it must be converted into a digital signal, which is a simple numeric representation of the magnitude of voltage or current of the original signal.

If we want to work with any of such analog signals like Temperature, Sound, Image, Pressure, etc. then, Analog to Digital Converters are the go-to devices. Almost all modern ADC implementations are IC based to reduce the complexity of matched components.

ADC in LPC1768 MCU

The LPC1768 MCU consists of an 8-channel 12-bit Successive Approximation type Analog to Digital Converter. Here, 8-channels indicate that you can convert 8 different analog signals on 8 different Analog Input pins of the MCU at a time.

12-bit is the resolution of the ADC, which represents the number of discrete values an ADC can produce for a given analog reference voltage. Resolution can also be represented electrically in terms of voltage, which is the smallest possible analog voltage an ADC can detect.

Since, the ADC in LPC1768 is having a resolution of 12-bits, it can produce 2^12 discrete values for a range of analog voltages, which is usually between 0V and 3.3V.

Resolution in terms of Voltage = $V_{REFP} – V_{REFN} / 2^M$

Where, V_{REFP} is the Positive Reference Voltage, which is 3.3V for LPC1768

V_{REFN} is the Negative Reference Voltage, which is 0V for LPC1768

M is the Resolution of ADC in bits, which is 12 for LPC1768.

Substituting these values, we get the Resolution in Volts as 0.000805V.

ElectronicsHub
(https://www.electronicshub.org)

Another important parameter of ADC is the Conversion Rate or Sampling Rate. The conversion rate of the ADC in LPC1768 MCU is 200kHz.

LPC1768 ADC Pins

Pins Related to ADC in LPC1768 MCU are listed in the following table.

Pin	***Port Pin***	When a Pin is used for ADC, its voltage shouldn't exceed VDDA. If pin is not used for ADC, then it is 5V tolerant.
AD0.0	P0.23	
AD0.1	P0.24	
AD0.2	P0.25	
AD0.3	P0.26	
AD0.4	P1.30	
AD0.5	P1.31	
AD0.6	P0.3	
AD0.7	P0.2	

ElectronicsHub (https://www.electronicshub.org)

	nd Negative Reference Voltages for both ADC and DAC (Isolated 3.3V and 0V)
V_{DDA}, V_{SSA}	ADC Power Pins (Isolated 3.3V and 0V)

LPC1768 ADC Registers

Now, let us take a look at the registers associated with ADC Module of LPC1768 MCU.

Register	***Function***	***Description***
ADCR	A/D Control Register.	Used to Start ADC, Select Operating Mode, Channel Selection, Set Clock Division.
ADGDR	A/D Global Data Register.	Contains the Data of the most recent A/D Conversion
ADINTEN	A/D Interrupt Enable Register.	Used to enable interrupt when a channel completes conversion.
ADDR0 – ADDR7	A/D Channel Data Register.	These registers hold the result of the recent conversion in that respective channel.
ADSTAT	A/D Status Register.	Holds the status of all ADC Channels.
ADTRM	A/D Trim Register.	Contains Trim values for ADC an DAC.

Please take a look at these registers in the reference manual. We will be using ADCR, ADGDR and ADDRx registers frequently.

ElectronicsHub
(https://www.electronicshub.org)

ADC Modes

Basically, there are two ways in which the ADC Module can work. One is the Software Controlled Mode and the other is Burst Mode (or Hardware Mode).

In Software Controlled Mode, only one channel must be made active during a conversion and at a time only one conversion is possible. To convert again, you have to repeat the process.

In Hardware Scan Mode or Burst Mode, the conversions will happen continuously on any number of channels starting from LSB (AD0.0) then progressing towards MSB (AD0.7).

Setup and Program ADC in LPC1768

Software-Controlled Mode

Let us now see the necessary steps to setup the ADC registers and initialize the ADC Conversion in software-controlled mode.

By default, some of the peripherals of LPC1768 are disabled by disabling their clocks. This is done to save power. In order to read from a peripheral register or write to peripheral register, you have to check whether the peripheral is enabled or not.

The Power Control for Peripherals Register or PCONP is used enable or disable a peripheral block. The ADC is disabled on reset. So, first enable ADC by setting the 12th bit in PCONP register.

```
LPC_SC->PCONP |= (1<<12);
```

ElectronicsHub
(https://www.electronicshub.org)

There is another power control for ADC in the ADCR register. By default, ADC is in power-down mode and this is controlled by the PDN (bit 21) of ADCR Register. To make ADC operational, we have to set the PDN bit.

```
LPC_ADC->ADCR |= (1<<21);
```

Using the ADCR register, we can also select the ADC Channel and also set the Clock Divider for ADC using SEL [7:0] and CLKDIV [15:8] bits.

We will select AD0.0 channel and set the ADC Clock Divider as 1.

```
LPC_ADC->ADCR |= (1<<0) | (1<<8);
```

Now, we can start the conversion immediately by setting START [26:24] bits in ADCR as 001 i.e. the 24th bit is set to 1.

```
LPC_ADC->ADCR |= (1<<24);
```

Now, we have to wait till the conversion is complete by monitoring the DONE (bit 31) bit in the corresponding ADDRx Register. Since we used AD0.0, we have to monitor the ADDR0 register and see if its DONE bit is set to 1,

```
while((LPC_ADC->ADDR0 & (1<<31)) == 0);
```

After completion, the result is stored in the ADDR0 register in its RESULT [15:4] bits. We can extract the result by shifting the ADDR0 bits by 4 places and masking the 12 bits.

```
int result = ((LPC_ADC->ADDR0 >> 4) & 0xFFF);
```

Burst Mode

Let us now see the necessary steps to setup the ADC registers and initialize the ADC Conversion in Burst mode.

The power setup in PCONP register and ADCR register and the clock divider settings remain the same even in burst mode.

ElectronicsHub (https://www.electronicshub.org)

```
LPC_SC->PCONP |= (1<<12);

LPC_ADC->ADCR |= (1<<21) | (1<<8);
```

Now, we can select more that one channels if we want by setting the corresponding SEL bits in ADCR [7:0]. Let us enable AD0.0 and AD0.1.

```
LPC_ADC->ADCR |= (1<<0) | (1<<1);
```

We can then start burst conversion by setting the BURST bit (bit 16) in ADCR Register to 1.

```
LPC_ADC->ADCR |= (1<<16);
```

We can obtain the channel number from which the ADC is converted as well as the result of ADC from the ADGDR Register.

If you take look at the ADGDR, then bits [26:24] contain the channel number and bits [15:4] contain the result. We can extract both as follows:

```
unsigned long valueADGDR = LPC_ADC->ADGDR;

int channel = ((valueADGDR >> 24) & 0x07);

int result = ((valueADGDR >> 4) & 0xFFF);
```

Conclusion

A simple tutorial on using ADC in LPC1768 MCU. I have explained the basic functionalities od ADC, its modes or operation, registers associated with ADC and programming setup for both software-controlled and burst mode ADC Conversion.

Related Posts:

- Getting Started with LPC1768 (https://www.electronicshub.org/getting-started-with-lpc1768/)
- How to Program GPIO in LPC1768? LPC1768 GPIO Tutorial (https://www.electronicshub.org/how-to-program-gpio-in-lpc1768/)
- How to use Timer in LPC1768? LPC1768 Timer Tutorial (https://www.electronicshub.org/how-to-use-timer-in-lpc1768/)
- Setting Up Keil MDK for LPC1768 (https://www.electronicshub.org/setting-up-keil-mdk-for-lpc1768/)
- How to Program UART in LPC1768? LPC1768 UART Tutorial (https://www.electronicshub.org/how-to-program-uart-in-lpc1768/)
- Basic Arduino Tutorials For Beginners (https://www.electronicshub.org/arduino-tutorial/)

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name *

Email *

Website

ElectronicsHub

(https://www.electronicshub.org)

Post Comment

About (https://www.electronicshub.org/about/)

Advertise with us

Contact (https://www.electronicshub.org/contact/)

Affiliate Disclosure(https://www.electronicshub.org/affiliate-disclosure/) | Disclaimer(https://www.electronicshub.org/disclaimer/) | Terms and Conditions(https://www.electronicshub.org/terms-and-conditions/) | Privacy Policy(https://www.electronicshub.org/privacy-policy/)

Copyright © 2024 Electronicshub.org